

Interactive Graphics Project Report

Development of an Interactive Open-World 3D Mystery Game

Mohamed Aziz Selmi

Degree Course: Artificial Intelligence and Robotics

Exam: Interactive Graphics

Academic Year: 2025–2026

Interactive Graphics
Academic Year 2025–2026

Contents

1	Introduction	3
2	Game Overview	3
3	Game Settings	3
3.1	Difficulty Settings	3
3.2	Character Customization	4
4	Game World	5
4.1	Casino	5
4.2	Garden	5
4.3	Road and Traffic Area	5
5	Technical Aspects	6
5.1	Textures	6
5.1.1	Casino Textures	6
5.1.2	Road and Urban Textures	6
5.1.3	Garden Textures	6
5.2	Lighting System	7
5.3	Shadow System	7
5.4	Camera System	7
5.5	Collision System	8
5.6	Hierarchical 3D Models	9
5.7	Animation Strategies	11
5.8	Vehicle Animation	12
5.9	Audio	13
5.10	External Libraries	13
5.11	External 3D Assets and Credits	14

1 Introduction

The aim of this project is to design and implement an interactive 3D open-world game that integrates exploration, interaction, and gameplay mechanics.

The game is based on a mystery investigation. The player is placed inside an explorable urban environment and is asked to investigate a jewelry store theft that has been causing serious problems in the neighborhood. The final objective is to collect enough information to identify and find the thief responsible for the crime.

The project focuses on creating an environment in which the player can freely explore different areas, interact with characters and objects, and progressively discover information related to the investigation.

2 Game Overview

To solve the case, the player is required to explore the environment, collect clues, and interact with different characters and objects.

Information obtained during the investigation gradually provides the player with the elements required to understand what happened and continue the mission.

The investigation is structured as a sequence of connected events. Certain interactions become relevant only after previous events have been completed.

The player therefore progresses through the game by exploring the environment and completing different interactions that gradually unlock new information and events. This structure connects the exploration of the open world with the main investigation and guides the player toward the final objective of identifying the thief.

3 Game Settings

Before entering the game world, the player can configure some aspects of the experience through the initial settings menu.

The game allows the player to select a difficulty level and customize the appearance of the main character.

These options allow the experience to be adapted to different players without changing the main structure of the investigation.

3.1 Difficulty Settings

The game can be played using three different difficulty levels: **Easy**, **Intermediate**, and **Hard**. The difficulty system does not modify the main investigation or the structure of the game world. Instead, it changes the amount of information and assistance provided to the player during the mission.

Easy difficulty provides the highest level of assistance. The player can clearly see the current objective and receives additional navigation support through the minimap, making it easier to identify important locations related to the investigation.

Intermediate difficulty provides a balance between assistance and independent exploration. The current objective remains available, but the player receives less direct navigation assistance and must understand where to go using the information obtained during the investigation.

Hard difficulty provides the lowest level of assistance. The player must rely more heavily on observation, dialogue, memory, and exploration. Direct guidance is reduced, requiring a better understanding of the environment and of the information collected during previous interactions.

The difficulty system is therefore mainly based on the amount of assistance available to the player. Increasing the difficulty reduces the amount of explicit guidance and gives greater importance to independent exploration and investigation.

3.2 Character Customization

The initial settings menu also allows the player to customize the appearance of the main character by selecting different clothes.

The available options modify the colors and visual appearance of the character's outfit while maintaining the same player model and animations.

This customization is exclusively visual and does not affect movement, interactions, collisions, or the progression of the investigation.

4 Game World

The game takes place in an explorable urban environment composed of different connected areas. Each area has its own visual identity, atmosphere, and role within the game.

The environment combines outdoor and indoor locations, allowing the player to move between different areas while following the investigation or freely exploring the game world.

4.1 Casino

The casino is one of the main indoor environments of the game and has a different atmosphere from the outdoor areas.

The player can enter the building, explore its interior, interact with characters, and use different objects and machines. The environment contains decorative elements, NPCs, doors, and interactive machines that make the location feel active rather than being only a visual area.

One of the main interactive elements is a claw machine. The machine combines an imported cabinet model with additional parts created directly in Three.js. The player can control the claw, lower it toward the toys, attempt to grab an object, and observe the result of the interaction.

The casino therefore works both as an explorable location and as a place in which different gameplay interactions are concentrated.

4.2 Garden

The garden is an outdoor area designed to provide a more elegant and relaxing environment within the game world.

It contains vegetation, rose gardens, a fountain, decorative elements, and different spaces dedicated to artistic exhibitions.

Within the garden, artists can exhibit their works, creating an area where the player can explore and observe different artistic pieces integrated into the surrounding environment.

This area combines nature, architecture, and art to create a visually distinct location compared with the other parts of the game world.

4.3 Road and Traffic Area

The road system connects the different areas of the environment and contributes to the open-world structure of the game.

Vehicles move through the environment using dedicated movement and collision systems, making the urban environment feel more dynamic and active.

5 Technical Aspects

This section describes the main technical components used to construct and manage the 3D environment.

The implementation combines textures, lighting, camera management, vehicle animation, collision detection, shadows, hierarchical models, animation strategies, external libraries, and three-dimensional assets in order to create the final interactive environment.

5.1 Textures

Textures are used throughout the game to provide visual detail to the different environments without requiring all surface information to be represented directly through geometry.

The project uses different textures depending on the type of environment and surface that has to be represented.

5.1.1 Casino Textures

The casino uses a dedicated floor texture stored as `floor.png`.

The texture is repeated across the floor surface instead of being stretched over the entire geometry. This allows the floor to maintain a consistent level of detail independently of its dimensions. The texture is associated with a `MeshStandardMaterial`, which also defines properties such as roughness and metalness.

5.1.2 Road and Urban Textures

The road surface uses an asphalt texture stored as `asp.jpg`.

This texture is applied to the road material in order to provide a more realistic representation of the street compared with a uniform color.

Concrete surfaces use the texture `concrete.jpg`. This material is used for urban surfaces and pedestrian areas that require a concrete appearance.

Both textures use repeated UV coordinates, allowing the same image to cover surfaces of different dimensions without excessive stretching.

5.1.3 Garden Textures

The outdoor and garden environments use `grass.png` to represent large vegetated surfaces.

The grass texture is configured using repeated wrapping and is repeated many times across the environment. This is particularly useful for large areas because a single texture can cover an extended surface while maintaining visible detail.

5.2 Lighting System

The lighting system combines global illumination with local light sources.

Instead of using a single light for the entire scene, different types of Three.js lights are used for different purposes.

An `AmbientLight` provides a basic level of illumination to the entire environment. This prevents surfaces that are not directly reached by another light source from becoming completely dark.

A `HemisphereLight` is also used to create a difference between light coming from the sky and light reflected from the ground. This provides a softer outdoor illumination.

A directional light is associated with the moon and represents a distant light source. Since a `DirectionalLight` produces parallel light rays, it is appropriate for representing illumination coming from a very distant source.

`PointLight` are used in locations where the illumination should originate from a specific position.

The project also manages local lights through a small reusable light pool. Only a limited number of actual point lights are used for rendering, while the system identifies the most relevant local light sources according to the current environment.

This approach reduces the number of active lights that have to be processed simultaneously and is useful for maintaining stable rendering performance in a large environment.

5.3 Shadow System

Shadows are used throughout the game to improve the perception of depth and to connect characters and objects more naturally with the environment. They contribute to the night atmosphere and make the effect of the different light sources more visible.

In the garden, shadows are produced through a **directional light**. This light illuminates the garden and creates visible shadows from the objects and structures present in the area, giving more depth to the outdoor environment.

The street lamps also contribute to the shadow system. They generate darker and more localized shadows around the illuminated areas, making their effect clearly visible during the night. Characters affected by these lights.

The game also provides a runtime option that allows the player to **enable or disable the shadows**. This option affects the shadow system of the game, allowing shadows to be activated for a more realistic visual result or disabled when desired.

5.4 Camera System

The game uses a third-person perspective camera implemented using `THREE.PerspectiveCamera`. The camera follows the player while maintaining a configurable distance, height, and viewing

target.

The desired camera position is calculated using the direction of the player and a lateral vector. The camera is therefore positioned behind the character while remaining oriented toward a point above the player's position.

The camera parameters are not identical in every environment. Different camera configurations are defined for outdoor and indoor areas. For example, the outdoor environment uses a greater camera distance and height, while indoor environments use a shorter distance in order to adapt to the reduced available space.

The movement of the camera is interpolated instead of being applied instantaneously. This creates a smoother transition when the player changes direction.

A particular system is also used inside the casino to prevent walls from covering the screen. Before placing the camera at its desired position, a ray is evaluated between the player and the camera. If structural geometry is detected between them, the camera distance is reduced.

5.5 Collision System

The game implements collision detection to prevent the player from passing through buildings, environmental objects, characters, vehicles, fences, and other solid elements.

The collision system distinguishes between static and dynamic colliders.

Static colliders represent elements whose position does not normally change, such as walls, fences, and parts of the environment. Dynamic colliders are updated during the game and are used for moving elements such as characters and vehicles.

Instead of using a complete physics engine or testing the exact geometry of every model, the project uses simplified geometric collision volumes.

Depending on the object, these volumes are represented using shapes such as boxes, cylinders, and other simple volumes that approximate the space occupied by the visible object. This solution keeps the collision system easier to control while still providing suitable accuracy for the game environment.

When the player attempts to move, the system first evaluates the desired new position. If no collision is detected, the movement is accepted. If the position is blocked, the system searches for the closest valid position between the previous and desired positions.

The use of simplified collision volumes provides a compromise between physical accuracy and real-time performance.



Figure 1: Visualization of the simplified collision volumes used in the environment.

5.6 Hierarchical 3D Models

Several objects used in the project are hierarchical models. In a hierarchical model, an object is composed of multiple connected elements organized using parent–child relationships. A transformation applied to a parent can therefore also affect the elements that belong to it. This structure is particularly important for articulated characters and for objects composed of several moving parts.

The main player character is loaded from `player.glb` and contains a hierarchical skeleton composed of multiple bones. The bones form a hierarchy representing the different parts of the body, including the torso, arms, forearms, hands, fingers, legs, knees, feet, and other joints. This structure allows individual parts of the character to be modified while maintaining the correct connection with the rest of the body.

For example, rotating an upper arm also changes the position of the forearm and hand because they are children of the same skeletal chain. In the same way, leg and foot movements can be combined to create a complete walking pose.

The other character models used in the game have a structure similar to `player.glb`. Since these characters come from the same original author and follow the same general organization,

the same type of bone and child-based animation logic can be reused for their movements and gestures.

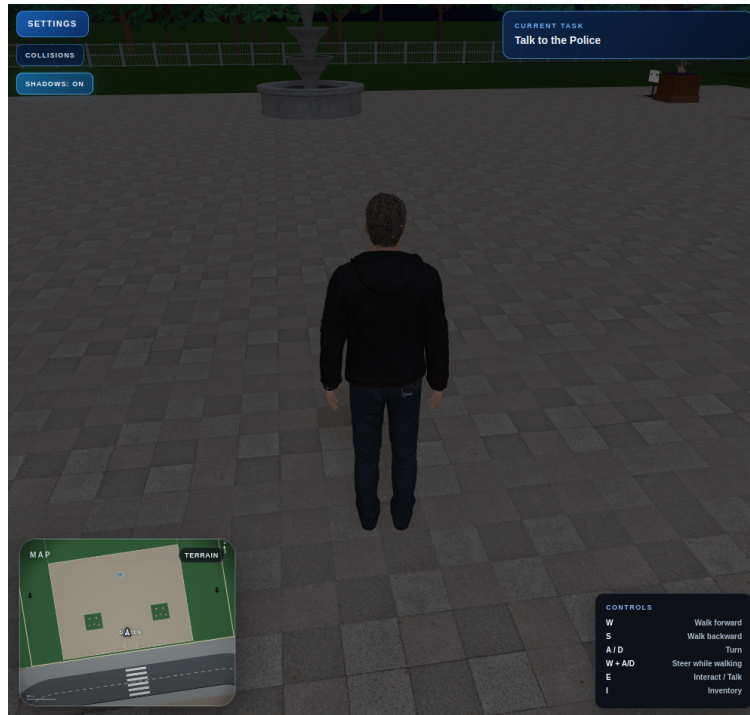


Figure 2: Player hierarchical model displayed in the garden environment.

The vehicle model, stored as `car.glb`, is also organized as a hierarchy composed of multiple child elements. The complete car can move and rotate as a single object while some of its internal parts, especially the wheels, can receive their own animation. This makes it possible to keep the vehicle aligned with the road while independently reproducing the rotation of the wheels.

The claw machine uses a mixed approach. The main cabinet is loaded as an external 3D model, while some functional elements are completed or modified directly in Three.js. In particular, the claw of the original model was replaced with a three-pronged claw. The cabinet, rod, claw and its prongs are handled as parts of the same hierarchy, allowing individual child elements to be moved independently while remaining connected to the complete machine.



Figure 3: Claw machine combining an imported model with additional Three.js elements.

5.7 Animation Strategies

The project uses more than one method to animate objects and characters, depending on how each model is structured and on the movement that has to be represented. Some movements are obtained by changing transformations over time, while others directly modify specific parts of a hierarchical model.

For smooth movements, the project uses **interpolation**. Instead of moving an object immediately from its current state to a new one, intermediate values are calculated between the two states. This is useful for rotations, changes of direction, camera movement, and transitions between character poses, because it avoids abrupt visual changes.

Characters also use **hierarchical animation**. Their models contain bones organized through parent–child relationships, so functions can modify selected parts of the body while the connected parts continue to follow the hierarchy. For example, the walking movement is produced by changing the rotations of the legs, knees, feet, arms, hands, and fingers over time. The two sides of the body move according to the same cyclic phase but in opposite directions, producing a continuous walking motion.

A similar idea is used for interactive objects composed of several child elements. In the claw machine, for example, the cabinet remains fixed while the rod, claw, and three prongs can be moved separately. Since these elements belong to the same hierarchy, each part can be controlled independently without losing its relationship with the complete machine.

Vehicle animation instead combines position updates with rotation. The car is moved along its trajectory while its orientation is progressively adjusted to follow the road. Internal child elements such as the wheels can also be animated independently from the movement of the complete vehicle.

These approaches allow the project to animate both complete objects and individual parts of hierarchical models without requiring every movement to be stored as a predefined animation sequence.

5.8 Vehicle Animation

Vehicles are animated independently from the player and contribute to making the road environment more dynamic. Their movement is updated continuously during the game loop by changing both their position and, when necessary, their orientation according to the trajectory they are following.

The vehicle movement can be represented using its position $P(t)$, where t represents the current time. At each update, the vehicle moves according to its velocity V , which represents both its direction of movement and its speed. The new position can therefore be expressed as:

$$P(t + \Delta t) = P(t) + V\Delta t$$

where Δt represents the time elapsed between two consecutive updates. This allows the movement to remain smooth and consistent over time.

When the vehicle follows a curved trajectory, its orientation is also updated so that it remains aligned with the direction of the road. The direction of the trajectory can be obtained from its tangent vector:

$$\vec{T}(t) = \frac{dP(t)}{dt}$$

where $\vec{T}(t)$ represents the direction in which the trajectory is moving at time t . This direction is then used to progressively rotate the vehicle and keep it aligned with the road while it moves. In this way, position and orientation are updated together, producing a continuous vehicle movement along both straight and curved sections of the road.

The vehicle can then be rotated according to this tangent so that its orientation remains aligned with the trajectory.



Figure 4: Vehicles following a curved section of the road.

5.9 Audio

Audio is used to support the atmosphere of the casino. The project includes two music tracks that can be activated by the player through the jukebox, which can only be used while the player is inside the casino.

The first track is **Cinematic Epic Ballad Backing Track in A Minor — 70 BPM** by **Guitar Lab Tracks**. The second track is **Metal Drum Track 150 BPM #2 (HQ,HD)**.

The jukebox allows the player to choose which of the two tracks to play while exploring the casino. The music is therefore controlled directly by the player and is available only inside the casino.

5.10 External Libraries

The project uses a small number of external libraries and utilities. Internal modules created specifically for the project are not considered external libraries in this section.

The main external graphics library is **Three.js**. It provides the fundamental components required to construct and render the 3D environment, including the scene graph, cameras, geometries, materials, textures, lights, mathematical utilities, and WebGL renderer.

GLTFLoader is used to load external models stored in GLTF or GLB format. The loader converts the contents of these files into a Three.js object hierarchy that can then be positioned, scaled, animated, or otherwise controlled by the game.

The project also uses **BufferGeometryUtils**, particularly the geometry merging functionality. Compatible static geometries can be combined into a single geometry in order to reduce the number of independent objects submitted to the renderer.

The external components used in the project are:

- **Three.js**: rendering and 3D scene management;
- **GLTFLoader**: loading GLTF and GLB models;
- **BufferGeometryUtils**: geometry processing and merging.

These components provide general graphical functionality, while the game-specific systems and logic are implemented separately as part of the project.

5.11 External 3D Assets and Credits

The game contains several external three-dimensional assets obtained mainly from Sketchfab. After being imported, the models were adapted to the needs of the project by adjusting their position, rotation, scale, materials, hierarchy, collisions, or interaction behavior where necessary.

The character models and the car model are credited to **Q.SARDOR**. The claw machine cabinet is based on the **Claw Machine** model by **Victoria Reinhold** on Sketchfab. For the project, the original claw was replaced with a custom three-pronged claw. The garden door model is credited to **Mehdi Shahsavan** on Sketchfab. Additional small decorative models were obtained from various other Sketchfab authors and adapted where necessary.

The external 3D assets used in the project are:

- **Character models**: Q.SARDOR – Sketchfab;
- **Car model**: Q.SARDOR – Sketchfab;
- **Claw Machine**: Victoria Reinhold – Sketchfab, modified in the project with a custom three-pronged claw;
- **Garden door model**: Mehdi Shahsavan – Sketchfab;
- **Additional small 3D models**: various authors – Sketchfab.